LAPORAN
WORKSHOP Mobile Web Server
(TIF330805)
Acara 17
SEMESTER



Nama Tugas
API CRUD DENGAN PHP NATIVE (ARSITEKTUR MVC, ROUTER, DAN FRONT CONTROLLER)

Nama:
Adam Ridho Saifulloh

NIM:
E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pada Acara 15 dan 16, API Data Mahasiswa dibangun dalam satu file mahasiswa.php yang memuat koneksi database, query SQL, validasi, dan pembentukan response sekaligus. Cara ini cukup untuk API kecil, tetapi sulit dirawat ketika jumlah resource bertambah, karena logika yang berbeda bercampur dalam satu file.

Acara 17 membangun API CRUD (Create, Read, Update, Delete) dengan PHP Native, yaitu PHP murni tanpa framework, menggunakan arsitektur MVC (Model-View-Controller). Model mengurus data dan query, Controller mengatur logika endpoint dan response, sedangkan Router memetakan URL ke Controller. Seluruh request masuk melalui satu titik, yaitu Front Controller public/index.php, dan class dimuat otomatis dengan spl_autoload_register().

Struktur ini menjadi fondasi untuk acara-acara berikutnya. Pada praktikum ini juga dikerjakan tugas praktik mandiri berupa resource baru (Buku), validasi NIM yang tidak boleh duplikat pada lapisan Model, serta dokumentasi endpoint beserta diagram alur Router, Controller, dan Model.

1.2 Rumusan Masalah

- Bagaimana menyusun struktur folder proyek berpola MVC pada PHP Native?
- Bagaimana membangun lapisan Model, Controller, Router, dan Front Controller untuk API CRUD?
- Bagaimana class dimuat otomatis dengan spl_autoload_register() dan semua request diarahkan ke satu titik masuk melalui .htaccess?
- Bagaimana menguji endpoint POST, GET, PUT, dan DELETE menggunakan Postman?
- Bagaimana menambah resource baru, validasi NIM duplikat, dan dokumentasi endpoint pada pola MVC yang sama?

1.3 Tujuan

- Memahami konsep API CRUD, PHP Native, dan arsitektur MVC beserta Front Controller.
- Membangun Model, Controller, Router, autoloader, dan Front Controller untuk resource mahasiswa.
- Menguji seluruh operasi CRUD menggunakan Postman dan menelusuri alur eksekusinya.
- Menyelesaikan tugas praktik mandiri: resource buku, validasi NIM duplikat, dan dokumentasi endpoint.

1.4 Manfaat

- Kode lebih terstruktur karena tanggung jawab dipisah per lapisan.

- Resource baru dapat ditambahkan dengan pola yang sama tanpa mengubah struktur dasar.
- Folder app/ berada di luar public/ sehingga kode sumber dan kredensial database lebih aman.
- Menjadi dasar pengembangan menuju framework dan proyek yang lebih kompleks.

BAB II LANDASAN TEORI

2.1 API dan JSON

API (Application Programming Interface) adalah sekumpulan aturan dan mekanisme yang memungkinkan dua aplikasi atau lebih bertukar data tanpa perlu mengetahui implementasi internal masing-masing. Pada pengembangan web modern, API umumnya memakai protokol HTTP dengan JSON sebagai format data karena ringan, mudah dibaca, dan didukung hampir semua bahasa pemrograman. Pada PHP, data diubah menjadi JSON dengan json_encode(), dan JSON dari client dibaca dengan file_get_contents('php://input') yang kemudian di-decode dengan json_decode().

2.2 PHP Native

PHP Native adalah penulisan kode PHP murni tanpa framework seperti Laravel, CodeIgniter, atau Slim. Pendekatan ini cocok untuk pembelajaran karena mahasiswa dapat memahami langsung alur API dari menerima request, memproses logika, mengakses database, sampai mengirim response, tanpa abstraksi tambahan dari framework.

2.3 API CRUD

CRUD adalah empat operasi dasar pengelolaan data. Pada API, setiap operasi dipetakan ke HTTP method seperti pada Tabel 2.1.

Operasi	Fungsi	HTTP Method
Create	Menambahkan data baru	POST
Read	Menampilkan data	GET
Update	Mengubah data	PUT/PATCH
Delete	Menghapus data	DELETE

Tabel 2.1 Operasi CRUD dan HTTP method

2.4 Arsitektur MVC

MVC (Model-View-Controller) adalah pola arsitektur yang memisahkan kode ke dalam tiga lapisan. Model bertanggung jawab atas data dan interaksi langsung dengan database. View bertanggung jawab atas tampilan yang disajikan kepada pengguna. Controller menerima request, menentukan Model yang diakses, lalu menentukan response yang dikembalikan. Pemisahan ini umumnya diterapkan pada pemrograman berorientasi objek (OOP), yaitu memandang tiap bagian sistem sebagai objek dengan data dan method sendiri. Pada API murni, View digantikan oleh response JSON; folder views/ disiapkan untuk acara berikutnya.

2.5 Autoloader, Router, dan Front Controller

- spl_autoload_register() mendaftarkan fungsi autoloader yang dipanggil otomatis oleh PHP ketika sebuah class dipakai tetapi belum dimuat, sehingga cukup menulis new NamaClass() tanpa require manual.
- Router membaca URL dan HTTP method, lalu meneruskannya ke method Controller yang sesuai.

- Front Controller adalah satu titik masuk tunggal (public/index.php) yang menerima seluruh request. File .htaccess mengarahkan semua request yang bukan file atau folder nyata ke index.php.

2.6 Alur Kerja API CRUD

1. Client mengirim request HTTP.
2. PHP menerima request.
3. PHP melakukan proses CRUD ke database.
4. Database mengembalikan hasil.
5. PHP mengubah hasil menjadi JSON.
6. Client menerima response JSON.

2.7 PDO dan Prepared Statement

PDO (PHP Data Objects) menyediakan antarmuka seragam untuk mengakses berbagai jenis database. Prepared statement memisahkan perintah SQL dari data yang dimasukkan pengguna melalui placeholder bernama (:nama) atau tanda tanya (?), sehingga nilai data tidak pernah disisipkan langsung ke teks SQL dan aplikasi terlindung dari SQL Injection.

2.8 HTTP Status Code

Kode	Arti	Digunakan pada
200 OK	Request berhasil	GET, PUT, DELETE berhasil
201 Created	Data berhasil dibuat	POST berhasil
400 Bad Request	Request tidak valid	Field kosong, JSON rusak, ID bukan angka
404 Not Found	Data/endpoint tidak ditemukan	ID tidak ada, resource tidak terdaftar
405 Method Not Allowed	Method tidak didukung	Method selain GET, POST, PUT, DELETE
409 Conflict	Data bentrok	NIM sudah terdaftar
500 Internal Server Error	Error pada server	Koneksi database atau query gagal

Tabel 2.2 HTTP status code yang digunakan

BAB III HASIL PRAKTIK

3.1 Alat dan Bahan

- Laptop/PC
- XAMPP (Apache, MySQL/MariaDB, phpMyAdmin) dengan PHP 8.2
- Visual Studio Code
- Git dan GitHub
- Web browser
- Postman
- Modul praktikum BKPM Workshop SI Web Server (TIF330805)

3.2 Tugas Praktik Mandiri

3.3.1 Tugas 1: Model dan Controller Resource Buku

Resource baru buku dibuat dengan pola MVC yang sama. Tabel buku ditambahkan ke database, lalu dibuat Model Buku.php dan BukuController.php (turunan BaseController), kemudian resource 'buku' didaftarkan pada peta Router ('buku' => 'BukuController'). Tidak ada perubahan lain pada Front Controller, autoloader, maupun .htaccess, yang menunjukkan bahwa pola MVC mudah dikembangkan.

SQL tabel buku:

```sql
DROP TABLE IF EXISTS buku;
CREATE TABLE buku (
  id INT AUTO_INCREMENT PRIMARY KEY,
  judul VARCHAR(150) NOT NULL,
  penulis VARCHAR(100) NOT NULL,
  penerbit VARCHAR(100) NOT NULL,
  tahun_terbit SMALLINT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO buku (judul, penulis, penerbit, tahun_terbit) VALUES
('Pemrograman Web dengan PHP', 'Rahmat Hidayat', 'Informatika', 2021),
('Basis Data Relasional', 'Sari Wulandari', 'Andi Offset', 2020);
```

File: app/models/Buku.php

```php
<?php
class Buku {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
```

```
    }

    public function getAll(): array {
        return $this->db->query('SELECT * FROM buku ORDER BY id DESC')-
>fetchAll();
    }

    public function find(int $id) {
        $stmt = $this->db->prepare('SELECT * FROM buku WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }

    public function create(array $data): string {
        $stmt = $this->db->prepare(
            'INSERT INTO buku (judul, penulis, penerbit, tahun_terbit)
             VALUES (:judul, :penulis, :penerbit, :tahun)'
        );
        $stmt->execute([
            ':judul'    => $data['judul'],
            ':penulis'  => $data['penulis'],
            ':penerbit' => $data['penerbit'],
            ':tahun'    => $data['tahun_terbit'],
        ]);
        return $this->db->lastInsertId();
    }

    public function update(int $id, array $data): void {
        $stmt = $this->db->prepare(
            'UPDATE buku SET judul = :judul, penulis = :penulis,
             penerbit = :penerbit, tahun_terbit = :tahun WHERE id = :id'
        );
        $stmt->execute([
            ':judul'    => $data['judul'],
            ':penulis'  => $data['penulis'],
            ':penerbit' => $data['penerbit'],
            ':tahun'    => $data['tahun_terbit'],
            ':id'       => $id,
        ]);
    }

    public function delete(int $id): void {
        $stmt = $this->db->prepare('DELETE FROM buku WHERE id = :id');
        $stmt->execute([':id' => $id]);
    }
}
```

File: app/controllers/BukuController.php

```php
<?php
class BukuController extends BaseController {
    private Buku $model;

    public function __construct() {
        $this->model = new Buku();
    }

    // GET /buku dan GET /buku?id=1
    public function index(): void {
        $id = $_GET['id'] ?? null;

        if ($id !== null && $id !== '') {
            $id = $this->requireId($id);
            $data = $this->model->find($id);
            if (!$data) {
                $this->fail(404, 'Data tidak ditemukan');
            }
            $this->json(200, ['success' => true, 'message' => 'Data berhasil
ditemukan', 'data' => $data]);
        }

        $this->json(200, ['success' => true, 'message' => 'Data berhasil
diambil', 'data' => $this->model->getAll()]);
    }

    // POST /buku
    public function store(): void {
        $data = $this->validate($this->input());
        $id = (int) $this->model->create($data);

        $this->json(201, [
            'success' => true,
            'message' => 'Data berhasil ditambahkan',
            'data'    => ['id' => $id] + $data,
        ]);
    }

    // PUT /buku?id=1
    public function update($id): void {
        $id = $this->requireId($id);
        if (!$this->model->find($id)) {
            $this->fail(404, 'Data tidak ditemukan');
        }
        $data = $this->validate($this->input());
        $this->model->update($id, $data);

        $this->json(200, [
            'success' => true,
            'message' => 'Data berhasil diperbarui',
```

```
            'data'    => ['id' => $id] + $data,
        ]);
    }

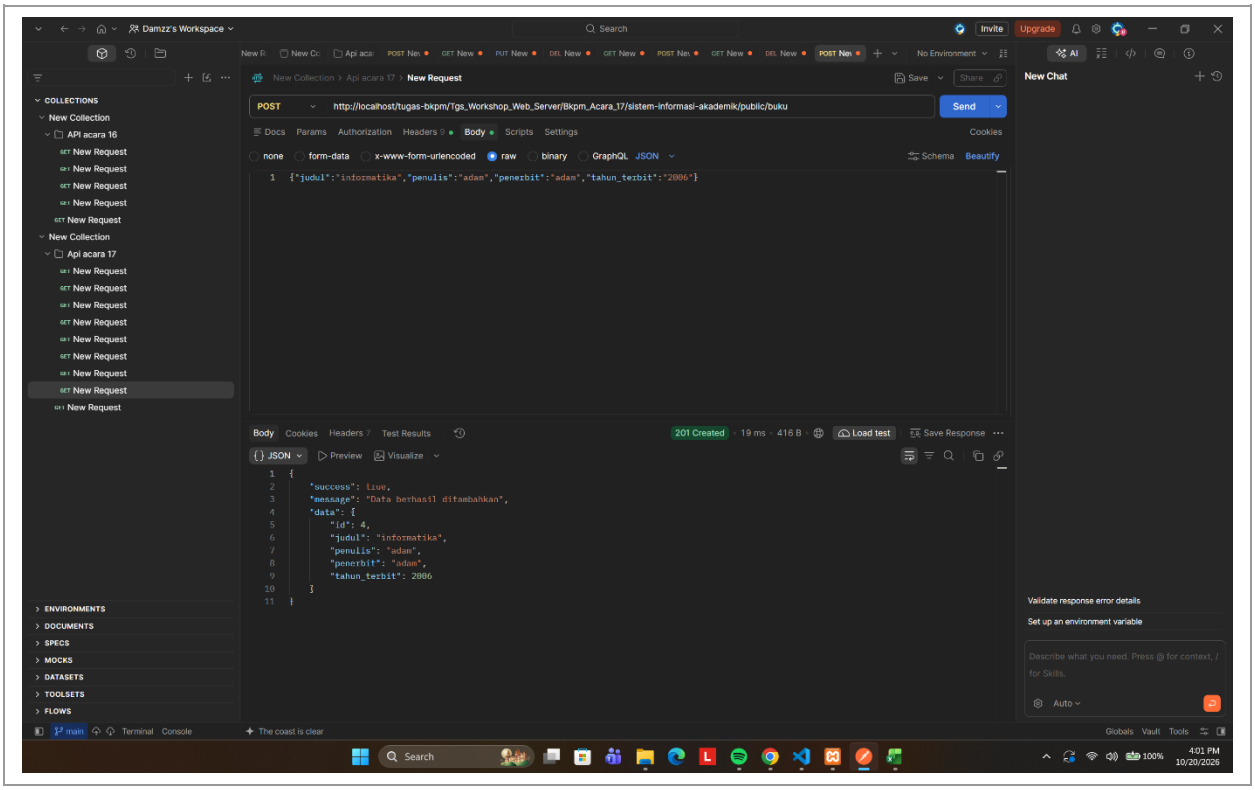
    // DELETE /buku?id=1
    public function destroy($id): void {
        $id = $this->requireId($id);
        if (!$this->model->find($id)) {
            $this->fail(404, 'Data tidak ditemukan');
        }
        $this->model->delete($id);
        $this->json(200, ['success' => true, 'message' => 'Data berhasil
dihapus', 'data' => null]);
    }

    private function validate(array $in): array {
        $judul    = $this->text($in['judul'] ?? '');
        $penulis  = $this->text($in['penulis'] ?? '');
        $penerbit = $this->text($in['penerbit'] ?? '');
        $tahun    = $this->text($in['tahun_terbit'] ?? '');

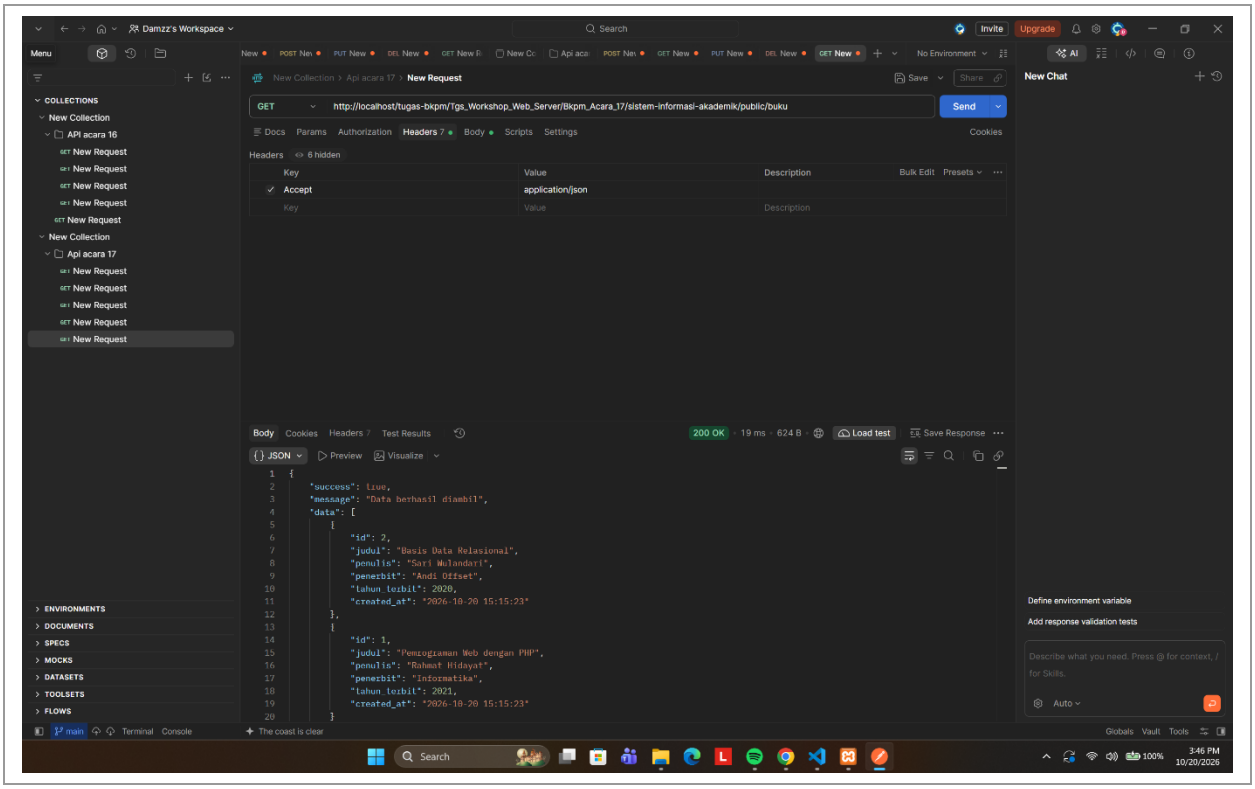
        if ($judul === '' || $penulis === '' || $penerbit === '' || $tahun
=== '') {
            $this->fail(400, 'Field judul, penulis, penerbit, dan
tahun_terbit wajib diisi');
        }
        if (!ctype_digit($tahun) || (int) $tahun < 1900 || (int) $tahun >
(int) date('Y') + 1) {
            $this->fail(400, 'tahun_terbit harus berupa angka tahun yang
valid');
        }
        if (strlen($judul) > 150)    $this->fail(400, 'Judul maksimal 150
karakter');
        if (strlen($penulis) > 100)  $this->fail(400, 'Penulis maksimal 100
karakter');
        if (strlen($penerbit) > 100) $this->fail(400, 'Penerbit maksimal 100
karakter');

        return [
            'judul'        => $judul,
            'penulis'      => $penulis,
            'penerbit'     => $penerbit,
            'tahun_terbit' => (int) $tahun,
        ];
    }
}
```

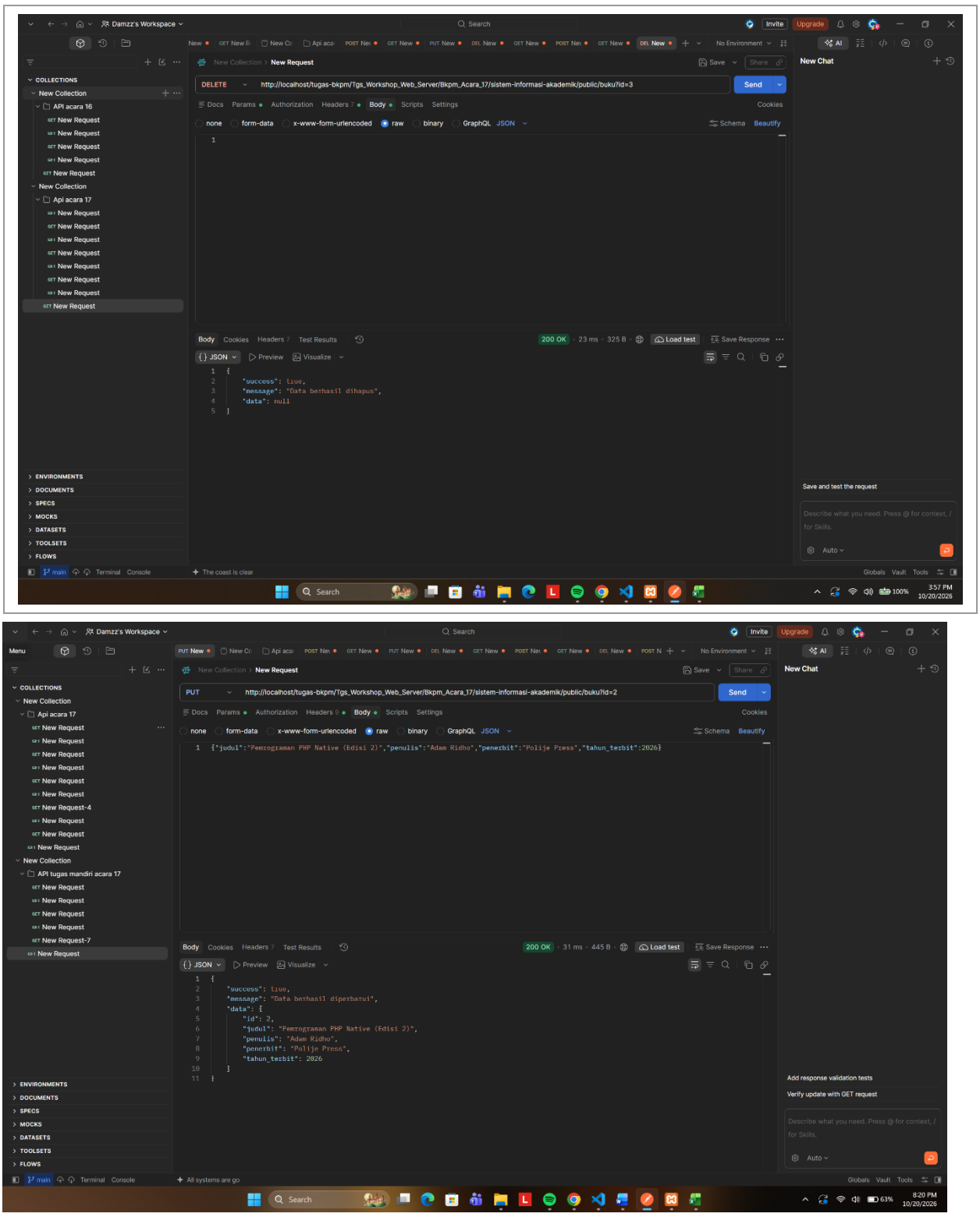
Pengujian: POST {{base_url}}/buku dengan body berikut menghasilkan status 201 Created.



Gambar 3.15. Pengujian POST /buku



Gambar 3.16. Pengujian GET /buku



Gambar 3.17. Pengujian PUT dan DELETE /buku

3.3.2 Tugas 2: Validasi NIM Duplikat pada Lapisan Model

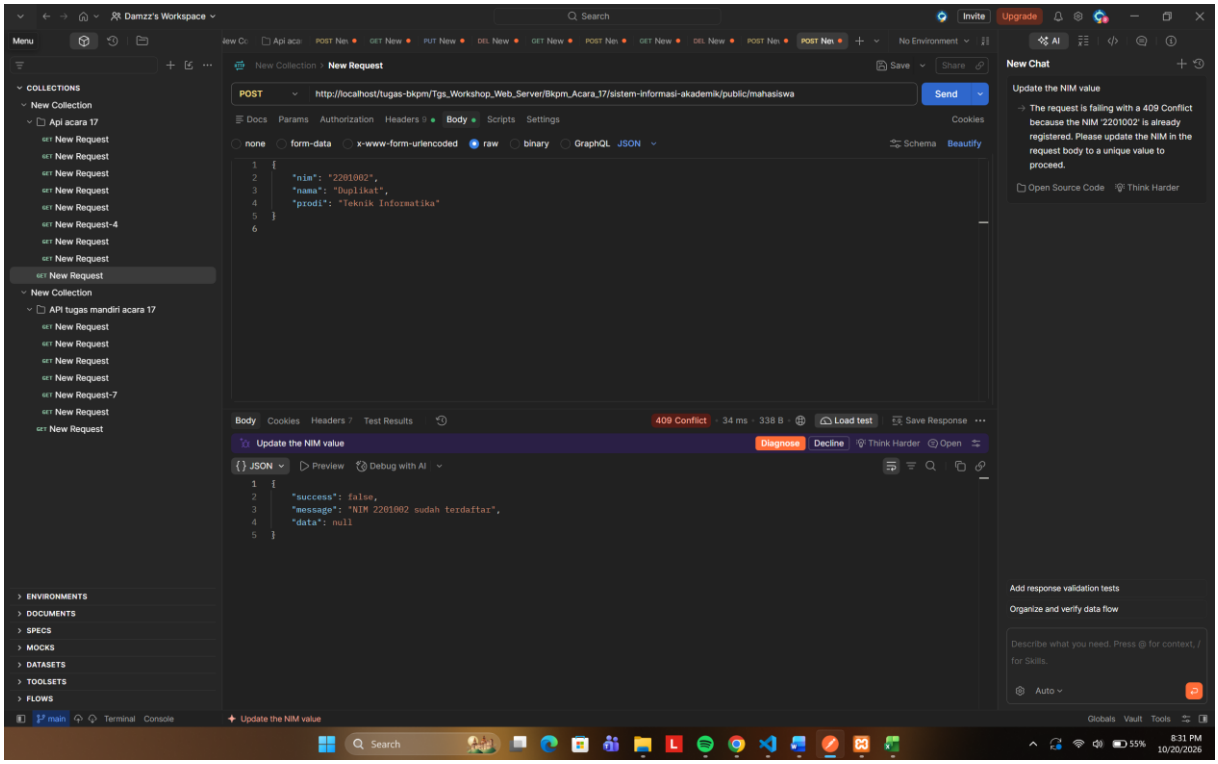
Validasi ditempatkan pada Model sesuai prinsip bahwa aturan data menjadi tanggung jawab Model. Method nimExists() menghitung jumlah mahasiswa dengan NIM yang sama menggunakan prepared statement. Pada update(), data dengan id yang sedang diubah dikecualikan (parameter $exceptId) agar mahasiswa dapat menyimpan data tanpa mengganti

NIM-nya. Jika NIM sudah ada, Model melempar DomainException; Controller menangkapnya dan menjawab status 409 Conflict. UNIQUE KEY pada database menjadi pengaman terakhir.

```
public function nimExists(string $nim, ?int $exceptId = null): bool {
        $sql = 'SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim';
        $params = [':nim' => $nim];
        if ($exceptId !== null) {
            $sql .= ' AND id <> :id';
            $params[':id'] = $exceptId;
        }
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);
        return (int) $stmt->fetchColumn() > 0;
    }

    public function create(array $data): string {
        if ($this->nimExists($data['nim'])) {
            throw new DomainException('NIM ' . $data['nim'] . ' sudah
terdaftar');
        }
```

Pengujian: POST dengan nim 2201002 (sudah dimiliki Siti Aminah) menghasilkan status 409 Conflict.



Gambar 3.19. Pengujian NIM duplikat

Pengujian PUT dengan NIM milik mahasiswa lain juga ditolak dengan status 409, sedangkan PUT yang memakai NIM miliknya sendiri berhasil (status 200).

3.3.3 Tugas 3: Dokumentasi Endpoint dan Diagram Alur

Dokumentasi endpoint disusun pada Tabel 3.2. Parameter id dikirim sebagai query string (?id=). Seluruh response berformat success, message, dan data.

Method	Endpoint	Fungsi	Status
GET	/mahasiswa	Semua mahasiswa	200
GET	/mahasiswa?id={id}	Satu mahasiswa	200, 400, 404
POST	/mahasiswa	Tambah mahasiswa	201, 400, 409
PUT	/mahasiswa?id={id}	Ubah mahasiswa	200, 400, 404, 409
DELETE	/mahasiswa?id={id}	Hapus mahasiswa	200, 400, 404
GET	/buku	Semua buku	200
GET	/buku?id={id}	Satu buku	200, 400, 404
POST	/buku	Tambah buku	201, 400
PUT	/buku?id={id}	Ubah buku	200, 400, 404
DELETE	/buku?id={id}	Hapus buku	200, 400, 404

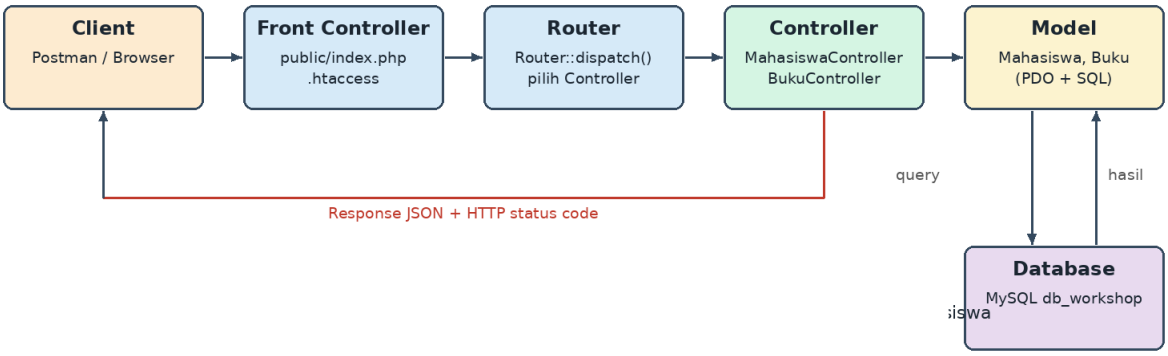
Tabel 3.2 Dokumentasi endpoint

Contoh request dan response untuk resource mahasiswa ditunjukkan pada Tabel 3.3 (body response selengkapnya terdapat pada subbab 3.2.9).

Endpoint	Contoh request	Contoh response
GET /mahasiswa	Tanpa body	200: success true, data berupa array mahasiswa (urut id terbaru)
GET /mahasiswa?id=4	Tanpa body	200: data satu mahasiswa; 404 jika tidak ada
POST /mahasiswa	{"nim":"2201001","nama":"Andi Saputra","prodi":"Teknik Informatika"}	201: message "Data berhasil ditambahkan", data memuat id baru
PUT /mahasiswa?id=4	{"nim":"2201001","nama":"Andi Saputra, S.Kom","prodi":"Teknik Informatika"}	200: message "Data berhasil diperbarui"
DELETE /mahasiswa?id=4	Tanpa body	200: message "Data berhasil dihapus", data null

Tabel 3.3 Contoh request dan response

Diagram alur Router, Controller, dan Model ditunjukkan pada Gambar 3.20. Request dari client masuk melalui Front Controller ke Router, diteruskan ke Controller yang sesuai, lalu Controller memanggil Model untuk mengakses database. Hasilnya dikembalikan sebagai response JSON. Sebagai contoh, POST /mahasiswa melewati Router::dispatch(), MahasiswaController::store(), Mahasiswa::create(), lalu perintah INSERT INTO mahasiswa.



Gambar 3.20. Diagram alur Router, Controller, dan Model

BAB IV KESIMPULAN

Berdasarkan praktikum Acara 17, dapat disimpulkan bahwa:

- API CRUD memetakan operasi Create, Read, Update, dan Delete ke HTTP method POST, GET, PUT, dan DELETE, dengan JSON sebagai format request dan response.
- Arsitektur MVC memisahkan tanggung jawab: Model mengurus data dan SQL, Controller mengatur logika endpoint dan response, dan Router memetakan URL ke Controller.
- Front Controller (public/index.php) dan .htaccess membuat seluruh request masuk melalui satu titik, sedangkan spl_autoload_register() memuat class secara otomatis tanpa require manual.
- Menempatkan folder app/ di luar public/ meningkatkan keamanan karena kode sumber dan kredensial database tidak dapat diakses langsung melalui browser.
- Resource buku berhasil ditambahkan hanya dengan membuat Model, Controller, dan mendaftarkannya pada Router, yang menunjukkan pola MVC mudah dikembangkan.
- Validasi NIM duplikat pada lapisan Model, ditambah penanganan status 400, 404, 405, 409, dan 500, membuat API memberi umpan balik yang jelas dan aman.

1.